

Relatório Técnico - Fase V

Alunos:

- Felipe Portes Antunes
- Lucas Teixeira Reis
- Thayná Andrade Caldeira Antunes

Professor: Walisson Ferreira de Carvalho

Link do Projeto: [Checklist App](#)

Link do vídeo desta etapa: [Casamento de padrões e criptografia](#)

1. Escolha do Campo Textual

Pergunta: Qual campo textual foi escolhido para aplicar os algoritmos de casamento de padrões? Por quê?

Resposta: Os algoritmos de casamento de padrões foram aplicados sobre os seguintes campos textuais do sistema:

1. **Tarefas (tarefa):** Campos titulo e descricao .
2. **Usuários (usuario):** Campos nome e email .

Justificativa: Em um aplicativo de gerenciamento de listas de tarefas (checklist), a pesquisa por título e descrição é a funcionalidade de busca mais crítica para o usuário final, permitindo localizar rapidamente atividades específicas no dia a dia. Já para usuários, a pesquisa por nome e e-mail no painel administrativo simplifica o gerenciamento de contas. A indexação e busca nestes campos de texto livre são cenários ideais para demonstrar o ganho de eficiência propiciado por algoritmos especializados de busca de padrões.

2. Algoritmo Knuth-Morris-Pratt (KMP)

Pergunta: Explique o funcionamento do KMP implementado.

Resposta: O algoritmo KMP realiza a busca de padrões em tempo linear $O(n + m)$ (onde n é o comprimento do texto e m o do padrão) sem retroceder o cursor de leitura do texto.

- **Pré-processamento:** O padrão é analisado para construir uma tabela de falhas chamada **LPS** (*Longest Proper Prefix which is also Suffix*). Essa tabela armazena o comprimento do maior prefixo próprio do padrão que também é um sufixo de sua subsequência.
- **Fase de Busca:** O texto é percorrido da esquerda para a direita. Quando ocorre um *mismatch* (não-casamento de caracteres), o KMP consulta a tabela LPS para determinar quantos caracteres do padrão podem ser pulados, reiniciando a comparação a partir da posição segura mais avançada do padrão sem retroceder no texto.

Código da Tabela LPS (KMPSearch.java):

```
private static int[] computeLPS(String pattern) {
    int[] lps = new int[pattern.length()];
    int length = 0;
    int i = 1;
    lps[0] = 0;

    while (i < pattern.length()) {
        if (pattern.charAt(i) == pattern.charAt(length)) {
            length++;
            lps[i] = length;
            i++;
        } else {
            if (length != 0) {
                length = lps[length - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}
```

3. Algoritmo Boyer-Moore (BM)

Pergunta: Explique o funcionamento do Boyer–Moore implementado.

Resposta: O algoritmo de Boyer-Moore compara o padrão com o texto da direita para a esquerda (começando do último caractere do padrão). Quando ocorre um mismatch, ele faz saltos maiores com base em duas heurísticas:

1. **Regra do Caractere Ruim (*Bad Character Heuristic*):** Ao falhar no caractere `c` do texto, procura-se a ocorrência de `c` à esquerda no padrão. Se existir, o padrão é deslocado para alinhar esses caracteres. Se não existir no padrão, o padrão inteiro é deslocado além de `c`.
2. **Regra do Sufixo Bom (*Good Suffix Heuristic*):** Caso um sufixo do padrão já tenha casado com o texto antes do mismatch, desloca-se o padrão para alinhar a próxima ocorrência desse mesmo sufixo (ou o maior prefixo dele que também seja sufixo).

O algoritmo calcula os deslocamentos sugeridos pelas duas regras e executa o salto de maior valor (`Math.max(badCharShift, goodSuffixShift)`).

4. Integração dos Algoritmos ao Sistema

Pergunta: Descreva como integrou os algoritmos ao sistema.

Resposta: A integração ocorreu de forma modular nas camadas backend e frontend:

1. **Backend (`SearchManager.java` & `SearchController.java`):**

 - A classe `SearchManager.java` atua como um serviço unificado que busca registros em memória persistidos em arquivos, utilizando a interface de algoritmo escolhida (KMP ou BM).
 - Registramos o `SearchManager` como Bean em `DaoConfig.java`.
 - Criamos o controlador REST `SearchController.java` na rota `/api/search` que recebe `pattern` e `algorithm` via parâmetros e retorna apenas as tarefas que combinam com o padrão e que pertencem ao usuário logado na sessão (ou todas, caso seja ADMIN).

2. **Frontend (`Dashboard.tsx`):**

 - Adicionamos um botão moderno “**Pesquisar Padrão (KMP/BM)**” no cabeçalho.
 - Implementamos um modal interativo onde o usuário seleciona o algoritmo (KMP ou Boyer-Moore), digita o padrão e clica em buscar.
 - Os resultados são retornados da API e destacados visualmente em tempo

real com marcações `<mark>` coloridas nas correspondências exatas em títulos e descrições das tarefas.

5. Dificuldades Encontradas

Pergunta: Quais dificuldades encontrou na implementação dos dois algoritmos?

Resposta:

- **Regra de Sufixo Bom:** A lógica matemática e de indexação para pré-processar a tabela de Good Suffix no Boyer-Moore é bastante sutil, especialmente o tratamento das bordas da string e correspondência de sufixos que saem das extremidades do padrão.
- **Depuração de Deslocamentos de Boyer-Moore:** Identificamos e corrigimos dois bugs críticos na lógica de deslocamento de Boyer-Moore original:
 - i. *Subtração Dupla do Bad Character:* A fórmula subtraía o retorno do método `getBadCharShift` (que já calculava a distância correta $j - last(C)$) de j , fazendo com que o algoritmo calculasse deslocamentos incorretos ou negativos, ignorando ocorrências válidas no texto (como a palavra “trabalho” precedida de outros caracteres).
 - ii. *Loop Infinito pós-Casamento:* Quando ocorria um casamento exato e `shift + m < n`, o padrão avançava `m - goodSuffixTable[0]`. No entanto, se essa diferença fosse `<= 0`, o algoritmo entrava em loop infinito e esgotava a pilha de memória (`OutOfMemoryError`). Forçamos um avanço mínimo de 1 caractere (`Math.max(1, nextShift)`) para evitar este travamento.
- **Segurança e Filtros de Sessão:** Garantir que as buscas no backend retornassem apenas tarefas do usuário logado na sessão atual, visto que os DAOs do sistema operavam no arquivo de persistência de baixo nível contendo registros de todos os usuários.
- **Marcação Dinâmica de Texto:** A divisão e destaque de texto no frontend React sem quebrar caracteres acentuados ou alterar maiúsculas/minúsculas das correspondências exigiu o uso de Expressões Regulares com grupos de captura apropriados.

6. Campo Utilizado na Criptografia

Pergunta: Qual campo foi utilizado na criptografia?

Resposta: Foi utilizado o campo de senha (`senha`) da entidade **Usuário** (`Usuario.java`), considerado o campo mais sensível e que exige proteção obrigatória contra vazamentos e leituras diretas em disco.

7. Método Utilizado na Criptografia

Pergunta: Qual foi o método utilizado na criptografia?

Resposta: Foi implementada a criptografia simétrica **XOR (OU Exclusivo)** na classe `XORCrypto.java` .

- **Processamento:** Cada byte da senha é combinado com o byte correspondente de uma chave de segurança secreta (`ChecklistApp2024!@#SecureKey`) por meio da operação lógica binária XOR ($\wedge$). A chave se repete ciclicamente caso a senha seja mais longa.
- **Armazenamento:** O array resultante é codificado em **Base64** para armazenamento seguro como texto legível no arquivo de persistência (`dados/usuarios.db`).
- **Reversibilidade:** Devido à propriedade involutiva da operação XOR ($A \oplus B \oplus B = A$), a descryptografia para validação do login utiliza exatamente o mesmo processo e chave, decodificando a string Base64 e aplicando novamente a operação binária XOR para recuperar a senha original.